

Consultation analysis: a production architecture

Thomas Butler

One central service that turns a department's consultation responses into signed-off themes and a filterable dashboard.

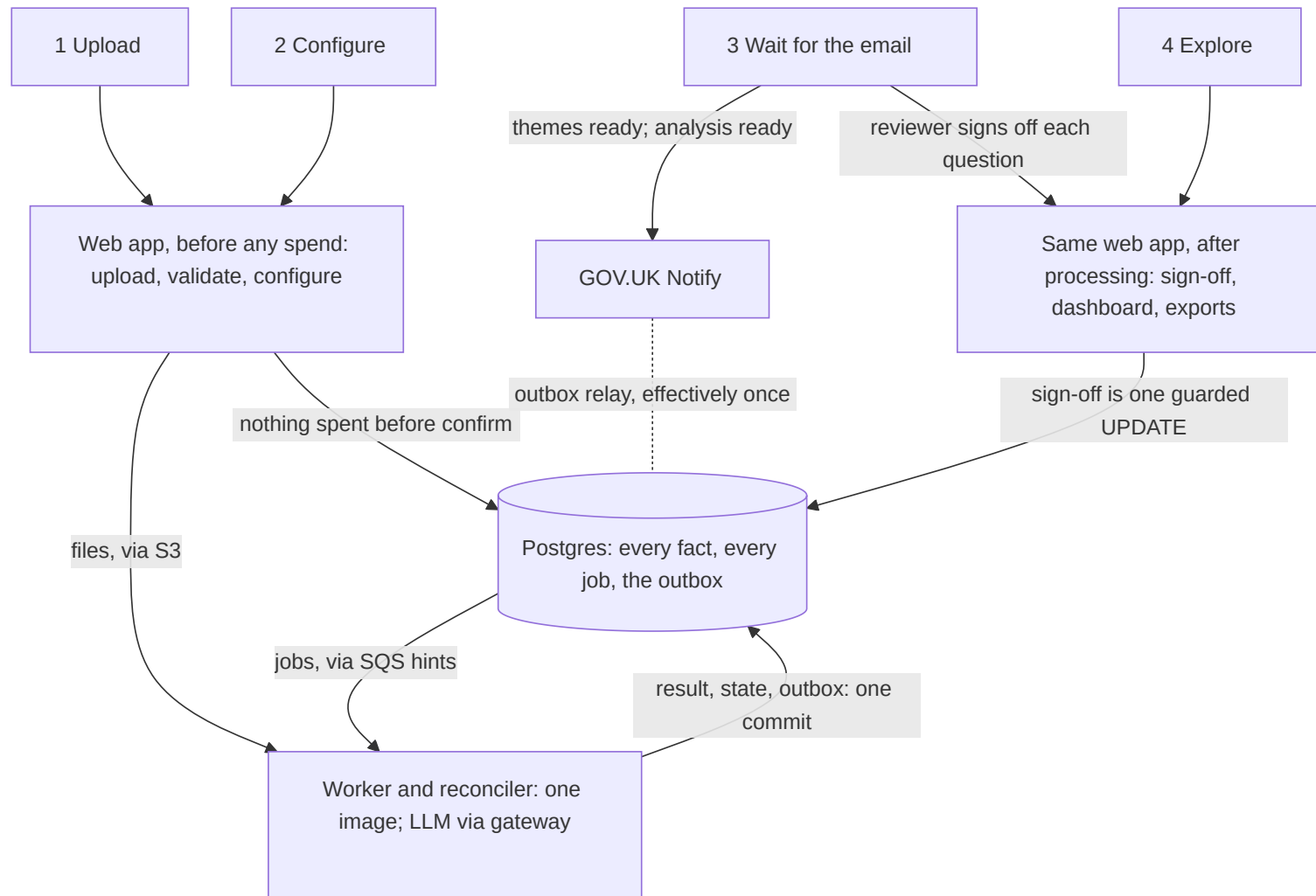


Diagram 1. One database holds every fact and job, reviewers sign off before tagging, and each email is written in the commit that earns it.

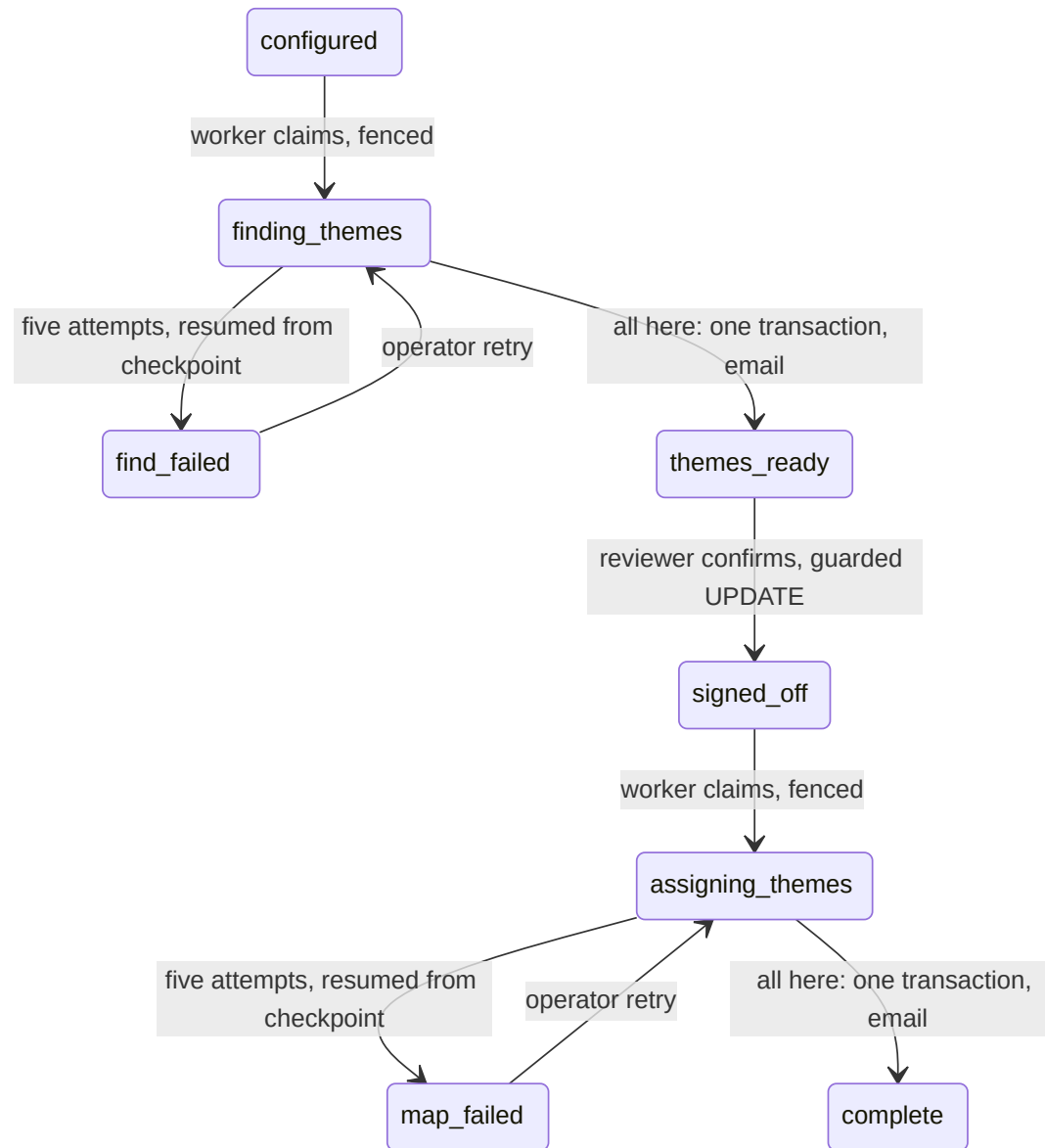


Diagram 2. Each question runs, fails and retries alone; awaiting_review, ready and both emails each come from a guarded UPDATE in the commit that finishes the last question of a phase.

The design, in bullets

- A named reviewer signs off each question's themes, and every edit is kept. An upload is validated and costed before anything is spent, and no demographic reaches a prompt. One service holds every department's data, scoped at the query and served fairly under caps.
- The journey gains one step: a reviewer signs off each question's themes before tagging, and the server enforces it. In i.AI's published evaluation, reviewers left about three mappings in four unchanged.
- Postgres holds every fact and all pipeline state; S3 holds only uploads and exports. The job table is the truth and queue messages are hints, so a duplicate or a lost message is harmless.
- One job per open question, with a lease, a fencing token and a checkpoint per batch: a dead worker's job resumes from its checkpoint, and a zombie can't write on a stale fence.
- The fan-in is one transaction behind a row lock: mark the question done, flip the consultation if every question is, queue the email. One finisher wins, and the reconciler covers a crash.
- One answer row per respondent, question and option, plus a respondent snapshot, so 'villagers who cycle to work and oppose' is one indexed query, measured at 20,000 rows once the index has settled after ingest. Tags are never deleted and theme lists are versioned, so counts reproduce.
- Responses are untrusted input: no tools, answers sent as JSON data, labels an enum of theme keys, small batches to bound the blast radius. Containment first; detection is a bonus.
- Encrypted at rest in London. Only the question, the open answer, any linked closed answer and the theme list reach the model, through a gateway. Each job's inference region belongs in the DPIA's record; the proof-of-concept doesn't store it yet.
- Parity first, with the manual process alongside and pilot consultations replayed as the gate. Then sign-off and the dashboard, then self-serve behind department caps. A small team, two quarters.
- Model spend is small beside reviewer time, as the water commission press release shows; throughput and sign-off time bind. Cost is a guard: an estimate before a run, which is built, and a department budget, which is designed.
- Least sure, so I'd test first: the dashboard's query plan under a selective filter on the largest consultations, and whether per-question sign-off is bearable at dozens of questions.

I read i.AI's public repository and decision records first; where 'every department' changes the problem I've gone further.

The brief asked for two hours. I chose to go further, because for a government service the foundations are where the judgement shows: the threat model, the guardrails, and a review log where every finding was verified before it counted. These three pages are the answer; the repository is the working.

I used Claude throughout: to research, to stress-test alternatives, to draft the diagrams and, under rules I set and a review pass before every merge, to write most of the proof-of-concept's code and tests. The choices are mine.